Practical 8

* Aim:- Implement session-based user login and logout functionality.

* Theory:- Session-based login and logout functionality is one of the most authentication mechanisms used in PHP web applications. A session allows the server to store user-specific information and maintain the user's state across multiple web pages. Since HTTP is a stateless protocol, sessions help identify a user after successful authentication.

In a login system, the user enters credentials such as a username and password through a form. PHP validates these credentials against predefined values or records stored in a database. If the credentials are correct, a session is started using session_start() and user information is stored in session variables. These session variables remains available until the user logs out or the session expires.

Ex:- Creating a session after successful login:-

```
<?php
    session_start();
    $username = "admin";
    $password = "1234";
    if ($_POST['username'] == $username &&
    $_POST['password'] == $password)
    {
```

```
    $_SESSION['user'] = $_POST['username'];
    echo "Login Successful";
}
?>
```

To protect pages from unauthorized access, the application checks whether the session variables exists before displaying content.

During logout, all session data is removed and the session is destroyed using session_destroy(). This ensures that the user must log in again to access protected pages.

Thus, PHP sessions provide a secure and efficient way to manage user authentication and maintain user login status throughout the application.

* Conclusion :- The session-based login and logout system was successfully implemented using PHP sessions. Sessions were used to maintain user authentication and securely manage access to protected pages. The logout features effectively ended the user session and removed stored & session data.

Practical 9

* Aim:- Create a simple Restful API in PHP to handle user information (GET, POST, PUT, DELETE requests).

* Theory:- A RESTful API (Representational State Transfer Application Programming Interface) is a web service that allows communication between applications using HTTP methods. In PHP, Restful APIs are commonly used to perform CRUD (Create, Read, Update, Delete) operations on data. The API receives requests from clients and returns responses, usually in JSON format. The main HTTP methods used in a RESTful API are:-

(i) GET - Retrieves users information from server.
(ii) POST - Adds a new user to server
(iii) PUT - Updates existing user information.
(iv) DELETE - Removes a user from the server.

PHP provides access to the request method through $_SERVER['REQUEST_METHOD']. Based on the request type, the appropriate operation is performed and a JSON response is returned to the client. RESTful APIs are widely used in web and mobile applications because they are lightweight, scalable and easy to integrate.

RESTful APIs typically exchange data in JSON (JavaScript Object Notation) format because it is light-weight, human-readable and supported by most programming languages. When a client sends a request, the server processes it, performs the required operations, and returns a JSON response containing the requested data or a status message.

ReST APIs help separate the frontend and backend of an application, making development and maintainance easier. They are commonly used in web applications, mobile applications, cloud services, and enterprise systems to provide secure and efficient access to data.

* Conclusion:- A simple RESTful API was successfully implemented in PHP to perform CRUD operations on user information. The API handled GET, POST, PUT and DELETE requests and facilitated efficient data exchange using JSON format.

PAGE NO.
ROLL NO.

Practical 10

* Aim:- Build a basic content management system (CMS) using PHP, MySQL and OOP principles.

* Theory:- A content Management System (CMS) is a software application that allows user to create, manage, edit, and delete digital content without requiring extensive programming knowledge. A CMS is commonly used for websites, blogs, news portals and other web applications where content needs to be uploaded frequently.

In this practical, a basic CMS is developed using PHP, MySQL and Object-Oriented Programming (OOP) principles. PHP is used as the server-side scripting language to handle application logic, while MySQL is used to store and manage content data. OOP concepts such as classes, objects, encapsulation, and inheritance help organize the code into reusable and maintainable components.

The CMS typically provides functionalities such as adding new content, viewing existing content, updating content, and deleting content. These operations are performed through user-friendly web interface and are connected to a MySQL database. OOP

improves code readability and scalability by separating database operations, business logic, and user interface components into different classes.

Using PHP with OOP principles makes the application more modular, reduces code duplication, and simplifies future enhancements. The integration of MySQL ensures efficient storage and retrieval of content, making the CMS suitable for managing dynamic website data.

* Conclusion :- A basic Content Management System (CMS) was successfully developed using PHP, MySQL, and OOP principles. The system enabled efficient content management through CRUD operations while demonstrating the benefits of OOP in building structured and maintainable web applications.